

1. Command line & Development Environment setup
2. Python basics, operators, calculations
3. All about strings
4. All about conditionals & loops
5. All about functions
6. List
7. Tuples
8. Dictionaries and data modeling
9. Sets
10. List Comprehension
11. Dictionary Comprehension and sets Comprehension
12. Advance flexible functions
13. Lambda expressions
14. Advance built-in functions
15. Decorators
16. Generators
17. OOP Basics
18. OOP Advance
19. Error Handling/Debugger

General Purpose Programming Language : Web, Desktop, Machine learning, Deep Learning, IE.

Download : Python application & Install First. **Compatible** : Win 7 Sp1 and Upper version.

Check : CMD open IDLE Check: `print(" Hello world")`

Other Editor: PyCharm, Jupyter, Visualstudio code (VSCODE), GitBash

code.visualstudio.com, gitforwindows.org

Install Git First restart and install VS Code.

=====

=====

`print("")` Outout Function

Escape sequences : `\' \' \' \n \t \b \\` Comment : use # shortcut ctrl+/
Exercise: Print Following statements as output.

```
1 # *****print these following lines *****
2 # this is \\ double backslash
3 # these are /\ /\ /\ /\ mountains
4 # he is      awesome (use escape sequence instead of manual spaces )
5 # \" \n \t \' (print this as an output)|
```

Solution:=====

```
print("this is \\\\ double backslash")
print("these are /\ /\ /\ /\ mountains")
print("he is\tawesome")
print(" \\\" \\n \\t \\\'| ")
# \" = "
# \\ = \
# \\\'| = \'
```

Apply also R For \n Print : <https://unicode.org/emoji/charts/full-emoji-list.html> ex: `U+1F601` `print("\U0001F601")`

=====

Operators: Arithmetic

Operators	Description	Example
+	Addition	>>> print(2+3) 5
-	Subtraction	>>> print(2-3) -1
*	Multiplication	>>> print(2*3) 6
/	Float division	>>> print(4/2) 2.0 >>> print(2/4) 0.5
//	Integer division	>>> print(4//2) 2 >>> print(2//4) 0
%	Modulo, it gives remainder	>>> print(6%2) 0
**	Exponent	>>> print(2**3) 8

Precedence rule

OPERATORS	PRECEDENCE AND ASSOCIATIVITY RULE
PARENTHESIS	HIGHEST
EXPONENT	RIGHT TO LEFT
*,/,//,%	LEFT TO RIGHT
+, -	LEFT TO RIGHT

This table lists the operator with highest precedence at the top and the lowest precedence at the top.

What is precedence rule?

To evaluate complex expression python follows the rule of precedence, it governs the order in which the operation take place.

```
print(2+3)
print(2+3*4)
print((2+3)*2)
print(2/4)
# floating point division print(2//4)
# integer division
print(2**3) # Exponent print(2**3**2)
```

Use Of Variable:

```
a =5
print(a)
a = "Nityam"
print(a)
print("Nityam" + "Webtech")
a = "Nityam"
b = "Webtech"
print(a + b), print(a + "3") ,print(a + str(3)) ,print(a * 3)
```

=====User Input=====

```
name = input("Enter Value")
print(name)
print("My Name is" + name)
```

```
number_one = int(input("enter first number ")) # 4
number_two = int(input("enter second number ")) # 4
total = number_one + number_two # 8
print("total is " + str(total))
number1 = str(4)
number2 = float("44")
number3 = int("33")
print(number2 + number3) # float
```

Two or more input in one line

```
# name = input("enter your name : ")
# age = input("enter you age : ")
```

```
name, age = input("enter your name and age ").split()
name, age = input("enter your name and age ").split(",")
```

```
print(name)
print(age)
```

Program:

```
num1 = input("enter first number :")
num2 = input("enter second number :")
num3 = input("enter third number :")
num1, num2, num3 = input("enter three numbers comma separated: ").split(",")
(int(num1) + int(num2) + int(num3)) / 3
print(f"average of three numbers : {(int(num1) + int(num2) + int(num3)) / 3}")
```

Flow Conrols:

If , If...else statement

```
# syntax
# if condition:
#     # code # if condition is true then this code will execute
#     # code
```

```
# pass statement
x = 18
if x > 18:
    pass
```

```
age = input("enter your age : ")
age = int(age)
if age >= 14:
    print("line a ")
    print("you are above 14")
else:
    print("sorry, you can't play ")
```

And or

```
# check two conditions at same time
# and , or
name = 'abc'
age = 19
if name == 'abcd' or age == 23:
    print("condition True")
else:
    print("condition False")
```

EXERCISE - WATCH Match

```
# Ask user's name and age
# If user's name start with ('a' or 'A') and age is above 10 then
# Print 'you can watch match'
# Else print 'sorry, you cannot watch match'
```

Answer:

```
user_name = input("enter your name please : ")
user_age = input("enter your age please : ")
user_age = int(user_age)
if user_age >= 10 and (user_name[0] == 'a' or user_name[0] == 'A'):
    print("you can watch match ")
else:
    print("you cannot watch match")
```

Nested If else

```
import random

lucky_number = (random.randint(1,101))
lucky_number = 27
user_input = input("guess a number b/w 1 and 100 : ")
user_input = int(user_input)

if user_input == lucky_number:
    print("YOU WIN !!!")
else: #nested if else
    if user_input < lucky_number:
        print("too low")
    else:
        print("too high")
```

If-elif-else statement

```
# if elif else statement

# show ticket pricing
# 1 to 3 (free)
# 4 to 10 (150)
# 11 to 60 (250)
# above 60 (200)

age = input("please input your age : ")
age = int(age)

if age==0 or age < 0:
    print("you can't watch")
elif 0<age<=3:
    print("Ticket price : Free")
elif 3<age<=10:
    print("Ticket price : 150")
elif 10<age<=60:
    print("ticket price : 250")
else:
    print("Ticket price : 200")
```

in keyword

```
name = "Nityam"
# in keyword
# if with in
if 'N' in "Nityam":
    print("H is present in name")
else:
    print("not present")
```

Exercise : Find letter if available than print position and count

Check Empty or Not

```
# check empty or not
# important
name = input("enter your name : ")

if name: # true if string is not empty
    print(f"your name is {name}")
else:
    print("you didn't type anything")
```

Loops:

While:

```
# loops
# while loop, for loop
# print("hello world") # 10 times

i = 2 # i = 2 , i = 3
while i<=10:
    print(f"hello world {i}")
    i = i + 1

# hello world
# hello world
# hello world
```

Other Example:

```
# sum : 1 to 10(or any number)
total = 0
i = 1 # i = 2
while i <= 20:
    total += i # total = total + i
    i += 1
print(total)
# total = 0 + 1 + 2 + 3 + 4
```

String formatting

```
name = "Nityam"
age = 24
print("hello " + name + " your age is " + str(age)) # ugly syntax
# string formatting
# python 3
print("hello {} your age is {} ".format(name, age)) # python 3

# 3.6
print(f"hello {name} your age is {age}") # clean
print(f"hello {name} your age is {age + 2}") # clean
```

String Indexing

```
# string indexing
language = "python"

# positions(index number)

# p = 0 , -6
# y = 1 , -5
# t = 2 , -4
# h = 3 , -3
# o = 4 , -2
# n = 5 , -1

print(language[-7])
```

String Slicing

```
# slicing / selecting sub sequences
lang = "Python"
# print(lang[4])
# syntax - [start argument : stop argument -1 ]
# print(lang[3:6])
# print(lang[-3:6])
# print(lang[:])
# print(lang[2:])
# print(lang[:4])
```

Step Argument

```
# slicing / selecting sub sequences
```

```
# lang = "Python"
# syntax - [start argument : stop argument -1 : step]
# print("Nityam"[0:5:2])
# print("Nityam"[0::2])
# print("Nityam"[:5:2])
# print("Nityam"[5::-1]) //reverse print
# print("Nityam"[-1::-1])
print("Nityam"[::-1]) # trick
```

Function & Method

```
name = "Nityam WebTEch"
```

```
1 len() Function
length = len(name)
print (length)
```

```
2 lower() Method
print (name.lower())
```

```
3 upper() Method
print (name.upper())
```

```
4 title() Method
print (name.title())
```

```
5 count() Method
print (name.count("E"))
```

Find and Replace Method

```
# replace() method
# find() method

string = "he is handsome and he is good dancer"
# print(string.replace("is", "was"))

# print(string.find("is"))
# print(string.find("is",1))
```

Program:exercise

```
# take two comma separated inputs from user
# 1.) user's name
# 2.) a single character

# output - 2 print lines
# 1.) user's name length
# 2.) count the character that user inputed (bonus : case insensitive count )
|
```

Answer:

```
name, char = input("enter comma separated name and character : ").split(",")
print(f"length of your name is {len(name)}")
print(f"character count : {name.count(char)}") # case sensitive
# NityamWebtEch - nityamwebtech
# E - e
#name.lower()
#char.lower()
#name.lower().count(char.lower())

print(f"character count : {name.lower().count(char.lower())}") # case Insensitive
```

Center Method

```
# # center method
# name = "nityam"
```

```
# # **nityam** , 10
# print(name.center(10,"*"))
Declare Two Variable and store two value:
name, age = "Nityam", "24"
print("hello " + name + " your age is " + age)
x=y=z=1
print(x+y+z)
```

Exercise: Enter Name and add 4 * both side
Sol:

```
name = input("enter your name : ")
print(name.center(len(name)+8, '*'))
```

Solve problem with spaces using strip method

```
# name = "      Nit          yam      "
# dots = "....."
# lstrip() method
# print(name + dots)
# print(name.lstrip() + dots)
# print(name.rstrip() + dots)
# print(name.strip() + dots)
# print(name.replace(" ", "") + dots)
```

Strings are Immutable

```
# strings are immutable
string = "string"
print(string.replace('t', 'T'))
print(string)
new_string = string.replace('t', 'T')
print(new_string)
```

Some More Exercise: strings with if ..else(Flow Control)

EXERCISE - WATCH Match

```
# Ask user's name and age
# If user's name start with ('a' or 'A') and age is above 10 then
# Print 'you can watch match'
# Else print 'sorry, you cannot watch match'
```

Answer:

```
user_name = input("enter your name please : ")
user_age = input("enter your age please : ")
user_age = int(user_age)
if user_age >= 10 and (user_name[0]=='a' or user_name[0] == 'A'):
    print("you can watch match ")
else:
    print("you cannot watch match")
```

Exercise:

```
# problem
# ask user to input a number containing more than one digit
# example - 1256
# calculate 1+2+5+6 and print
```

```
# algorithm - (method to solve problem in human language)
# ask input in string, i.e don't change string to int
# example - "1256"
# pick string character one by one and change to int
# int(example[0]) + int(example[1]) .... go upto len(example) - 1
```

Solutions:

```
number = input("enter a number ")
# "1256" # length = 4 , last index = 3
```

```
# int(number[i])

total = 0
i = 0
while i < len(number):
    total += int(number[i])
    i += 1
print(total)
Solutions:
number = input("enter a number ")
# "1256" # length = 4 , last index = 3
# int(number[i])
```

```
total = 0
i = 0
while i < len(number):
    total += int(number[i])
    i += 1
print(total)
```

```
Exercise:
# ask a user for name
# Example - Nityam Webtech
# print count of each letter
# output :
# N : 1
# i : 2
# t : 1
# y : 3
# a : 3
# m : 2
#   : 2
# w : 1
# e : 1
```

```
Solutions:
i = 0
while i < len(name):
    print(f"{name[i]} : {name.count(name[i])}")
    i += 1
-----
temp_var = ""
i = 0
while i < len(name):
    if name[i] not in temp_var:
        temp_var += name[i]
        print(f"{name[i]} : {name.count(name[i])}")
    i += 1
```

```
=====
# infinite loop
```

```
while True:
    print("hello world")
=====
```

For:

```
# i = 0

# while i <= 10:
#     print("hello world")
#     i = i + 1

# for loop with range function
for i in range(10):
```

```

    print(f"hello world : {i}")
    print("this is line \n ")
-----
for i in range(1, 11):
    print(f"hello world : {i}")
    print("this is line \n ")
-----
for i in range(1, 11):
    print(i,end = "")
Example:
# sum from 1 to 10
# 1 + 2 +3 ..... 10

# total = 0          # + 1 + 2 + 3 + 4 + ..... 20
# for i in range(1,21):
#     total += num

# print(total)

# n = int(input("enter the number : "))
# total = 0
# for i in range(1,n+1):
#     total += i
# print(total)
Example:
# practice for loop
# ask user a number like 1256
# caculate sum of digits ---> 1 + 2 + 5 + 6
# logic
# num = "1256" , length = 4
# int(num[0]) ---> 1
# int(num[1]) ----> 2
# int(num[2]) ----> 5
# int(num[3]) ----> 6
# i ---> 0 to 3

total = 0
num = input("enter a number : ")
for i in range(0, len(num)):
    total += int(num[i])

print(total)
Example: # practice for loop # ask user name and count each character

name = input("enter your name : ")
temp = ""
for i in range(len(name)):
    if name[i] not in temp:
        print(f"{name[i]}: {name.count(name[i])}")
        temp += name[i]
-----

```

Break and Continue keyword

```

# break and continue keyword

# 1 to 10 print
# for i in range(1, 11):
#     if i == 5:
#         break
#     print(i)

# continue
# print 1 to 10 , but not 5

```

```
# 1,2,3,4,6,7,8,9,10
for i in range(1,11):
    if i == 5:
        continue
    print(i)
# break and continue keyword

# 1 to 10 print
# for i in range(1, 11):
#     if i == 5:
#         break
#     print(i)

# continue
# print 1 to 10 , but not 5
# 1,2,3,4,6,7,8,9,10

for i in range(1,11):
    if i == 5:
        continue
    print(i)
```

Step argument in range function

```
# for i in range(0,11,2):
#     print(i)

# 10 , 9 , 8
# for i in range(10,0,-1):
#     print(i)

# for i in range(1, -11, -1):
#     print(i)
```

Random Input:

```
import random
winning_number = random.randint(1,100)
print (winning_number)
```

Exercise: Guess the number

Solution:

```
winning_number = 43
guess = 1
number = int(input("guess a number between 1 and 100 : "))
game_over = False

while not game_over:
    if number == winning_number:
        print(f"you win, and you guessed this number in {guess} times ")
        game_over = True
    else:
        if number < winning_number:
            print("too low ")
            guess += 1
            number = int(input("guess again : "))
        else :
            print("too high")
            guess += 1
            number = int(input("guess again : "))
```

```
import random
winning_number = random.randint(1,100)
guess = 1
number = int(input("guess a number between 1 and 100 : "))
```

```

game_over = False

while not game_over:
    if number == winning_number:
        print(f"you win, and you guessed this number in {guess} times ")
        game_over = True
    else:
        if number < winning_number:
            print("too low ")
        else :
            print("too high")

        guess += 1
        number = int(input("guess again : "))
        # DRY - don't repeat yourself

```

```

import random
winning_number = random.randint(1,100)
guess = 1
game_over = False

while True:
    number = int(input("guess a number between 1 and 100 : "))

    if number == winning_number:
        print(f"you win, and you guessed this number in {guess} times ")
        break
    else:
        if number < winning_number:
            print("too low ")
        else :
            print("too high")

        guess += 1
        continue
        # DRY - don't repeat yourself

```

For loop and string

```

# name = "nityam"
# for i in range(len(name)):
#     print(name[i])

# name = "nityam"
# for i in "nityam":
#     print(i)

# 1256 --- > 1+2+5+6

num = input("enter a number :")
total = 0
for i in num:
    total += int(i)
print(total)

```

User Defined Functions

```

# functions
# name = "nityam"
# print(len(name))

def add_two(num1,num2):
    return num1 + num2

# a = int(input("enter first number : "))
# b = int(input("enter second number : "))

```

```

# total = add_two(a,b)
# print(total)

first_name = input("type first name ")
last_name = input("type last name ")
print(add_two(first_name,last_name))
-----

# return vs print
def add_three(a,b,c):
    print( a+b+c)

add_three(5,5,5)
-----

# functions practice :
# def last_char(name):
#     return name[-1]

# print(last_char("Nityam Webtech"))
# # last_char(9) # error

# def odd_even(num):
#     if num%2 == 0:
#         return "even"
#     else:
#         return "odd"

# def odd_even(num):
#     if num%2 == 0:
#         return "even"
#     return "odd"

# print(odd_even(9))

# def is_even(num):
#     if num%2 == 0:
#         return True

#     return False

# def is_even(num):
#     return num%2 == 0 # true

# print(is_even(10))

# def hello():
#     return "I am here"

# print(song())
Example:Find Greater Number from two number:
def greater(a,b):
    if a > b:
        return a
    else:
        return b

num1 = int(input("enter first number "))
num2 = int(input("enter second number "))
bigger = greater(num1, num2)

print(f"{bigger} is greater")
Exercise:Find Greater Number from three number:
def greatest(a,b,c):
    if a>b and a>c:
        return a
    elif b>a and b>c:
        return b

```

```

    else:
        return c

print(greatest(100,40,30))

```

Function inside function

```

def greater(a,b):
    if a > b:
        return a
    return b

def new_greatest(a,b,c):
    bigger = greater(a,b)
    return greater(bigger, c)

#return greater(greater(a,b), c)

print(new_greatest(1000,200,30))

```

```

-----
Exercise : Find number palindrome or not
# def is_palindrom(word):
#     reversed_word = word[::-1]
#     if word == reversed_word:
#         return True
#     else:
#         return False

# def is_palindrom(word):
#     if word == word[::-1]:
#         return True
#     return False

```

```

def is_palindrom(word):
    return word == word[::-1]

print(is_palindrom("naman"))
print(is_palindrom("nityam"))

```

```

-----
Exercise : Print fibonacci series
# fibonacci series
# 0 1 1 2 3 5 8 13 21 34

# 1 ---> 0
# 2 ----> 0 1
# 3 ----> 0 1 1
# for i in range(1,11):
#     print(i, end = " ")

```

```

def fibonacci_seq(n):
    a = 0 # first number
    b = 1 # second number
    if n == 1:
        print(a)
    elif n == 2:
        print(a, b)
    else:
        print(a,b, end = " ")
        for i in range(n-2):
            c = a + b
            a = b
            b = c
            print(b , end = " ")

```

```

# fibonacci_seq(2)
-----

```

Default parameters

```
def user_info(first_name = 'unknown', last_name = 'unknown', age = None):
    print(f"Your first name is {first_name}")
    print(f"Your last name is {last_name}")
    print(f"you age is {age}")
```

```
user_info()
```

scope

```
x = 5 # global variable
```

```
def func():
    global x
    x = 7 # local variables
    return x
```

```
print(x)
print(func())
print(x)
```

Intro to list

```
# list vs array
# c , c++ , java
# array int, string

# list - store any data / flexible
# python array module - fix data type
# numpy arrays - binding with c libraries
```

```
-----
# data structures
# ordered collection of items

# you can store anything in lists int, float, string
numbers = [1, 2, 3, 4]
print(numbers)
```

```
words = ["word1", "word2" , 'word3']
print(words)
```

```
numbers = [1, 2, 3, 4]
print(numbers[1])
```

```
words = ["word1", "word2" , 'word3']
print(words[:2])
```

```
mixed = [1, 2, 3, 4 , "five", "six", 2.3, None]
print(mixed[-1])
```

```
mixed[1] = 'two'
print(mixed)
```

```

mixed[1:] = 'two'
print(mixed)

mixed[1:] = ['three', 'four']
print(mixed)
-----
-----

```

Add data to list

```

# how to add items to our list
# most common thing that you can do with your list and most important

# course = ['java', 'php']
# course.append('cpp')

# print(course)

# course = []
# course.append("cpp")
# course.append("java")
# print(course)
Exercise: Take five name dynamically and make list
user_names = []

for i in range(10):
    name = input("enter name : ")
    user_names.append(name)

```

More methods to add data

```

# some more methods to add data in our list
# insert method
# how to join(concatenate) two list
# extend method
# difference between append and extend method

course1 = ['cpp', 'html']
# course1.insert(1, "java")
# print(course1)
course2 = ['java', 'php']
# course = course1 + course2
# print(course)
course1.extend(course2)
# course1.append(course2)
print(course1)
print(course2)

```

Delete data from list

```

# common methods to delete data from list
fruits = ['orange', 'apple', 'pear', 'banana', 'apple', 'kiwi']
# pop method
# fruits.pop(1)

# del
# del fruits[1]

# remove
# fruits.remove('apple')

# append , extend , insert
# pop, remove, del

print(fruits)

```

In keyword with list

```

fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi']

```

```
# if 'mango' in fruits:
#     print("mango is present ")
# else :
#     print("not present")
```

Some more list methods

```
# count
# sort method
# sorted function
# reverse
# clear
# copy
fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana']
# print(fruits.count('orange'))
# fruits.sort()
numbers = [3, 5, 1, 9, 10]
# numbers.sort()
# print(sorted(numbers))
# numbers.clear()
numbers_copy = numbers.copy()
print(numbers_copy)
```

is vs equals Difference

```
# compare lists
# == , is
fruits1 = ['orange', 'apple', 'pear']
fruits3 = ['orange', 'apple', 'pear']
fruits2 = ['banana', 'kiwi', 'apple', 'banana']
print(fruits1 == fruits3) # values are same
print(fruits1 is fruits3) # false
```

join and split method

```
# split method
# convert string to list
# user_info = 'nityam 24'.split()
# print(user_info)
# user_info = 'nityam, 24'.split(',')
# print(user_info)
# name, age = 'nityam, 24'.split(',')
# print(name)
# print(age)

# name, age = input("enter your name and age ").split(',')
# print(name)
# print(age)

# join method
# convert list to string
# user_info = ['nityam', '24']
# print(','.join(user_info))
```

List vs String

```
# list vs strings
```

```
# strings are immutable
# lists are mutable

s = "string"
s.title()
print(s)

# t = s.title()
# print(t)

l = ['word1', 'word2', 'word3']
l.pop()
print(l)

l.append("word3")
print(l)
```

Looping in list

```
fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi']

# for loop
# for fruit in fruits:
#     print(fruit)

# while
# i = 0
# while i < len(fruits):
#     print(fruits[i])
#     i +=1
```

List inside List

```
# list inside list
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # 2d list
# 3 items --> 3 list
# print(matrix[2])

for sublist in matrix:
    print(sublist)

# for sublist in matrix:
#     for i in sublist:
#         print(i)

# print(matrix[2][0])

s = "string"
# print(type(s))
# print(type(matrix))
mixed = [1, 2, 'word', [4, 5, 6], [7, 8, 9]]
```

More about lists

```
# generate lists with range functions
# something more about pop method
# index method
# pass list to a function

# numbers = list(range(1,11))
print(numbers)
```

```

numbers.pop()
print(numbers)

print(numbers.pop())

# popped_item = numbers.pop()
# print(numbers)

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 5, 7, 8, 1]
print(numbers.index(1))
print(numbers.index(1, 3))

# print(numbers.index(1, 11, 14))

```

Exercise1

```

# define a function which will take list containing numbers as input
# and return list containing negative of every elements

```

Answer:

```

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 5, 7, 8, 1]

def negative_list(l):
    negative = []
    for i in l:
        negative.append(-i)
    return negative

print(negative_list(numbers))

```

Exercise2

```

# define a function which will take list containing numbers as input
# and return list containing square of every elements

```

```

# example
# numbers = [1, 2, 3, 4]
# square_list(numbers) ----> return ----> [1, 4, 9, 16]

```

Answer:

```

def square_list(l):
    square = []
    for i in l:
        square.append(i**2)
    return square

# list, str
numbers = list(range(1, 11))
print(square_list(numbers))

```

Exercise3

```

# define a function which will take list as a argument and this function
# will return a reversed list

```

```

# examples
# [1, 2, 3, 4] ---> [4, 3, 2, 1]
# ['word1', 'word2'] ---> ['word2', 'word1']

```

```

# Note you simply do this with reverse method or[::-1]

```

```

# but try to do this with the help of append and pop method

```

Answer:

```

# def reverse_list(l):
#     l.reverse()
#     return l

# def reverse_list(l):
#     return l[::-1]

```

```
def reverse_list(l):
    r_list = []
    for i in range(len(l)):
        popped_item = l.pop()
        r_list.append(popped_item)
    return r_list
```

```
numbers = [1,2,3,4]
print(reverse_list(numbers))
```

Exercise4

```
# define a function that take list of words as argument and
# return list with reverse of every element in that list
```

```
# example
# ['abc', 'tuv', 'xyz'] ---> ['cba', 'vut', 'zyx']
```

Answer:

```
def reverse_elements(l):
    elements = []
    for i in l:
        elements.append(i[::-1])
    return elements
```

```
words = ['abc', 'tuv', 'xyz']
print(reverse_elements(words))
```

Exercise5

```
# filter odd even
```

```
# define a function
# input
# list ---> [1,2,3,4,5,6,7]
```

```
# ouput ----> [[1,3,5,7], [2,4,6]]
```

Answer:

```
def filter_odd_even(l):
    odd_nums = []
    even_nums = []
    for i in l:
        if i % 2 == 0:
            even_nums.append(i)
        else:
            odd_nums.append(i)
    output = [odd_nums, even_nums]
    return output
```

```
nums = [1,2,3,4,5,6,7]
print(filter_odd_even(nums))
```

Exercise6

```
# common elements finder function
# define a functions which take 2 lists as input and return a list
# which contains common elements of both lists
```

```
# example
# input ---> [1,2,5,8], [1,2,7,6]
# output ---> [1,2]
```

Answer:

```
# example
# input ---> [1,2,5,8], [1,2,7,6]
# output ---> [1,2]
```

```
# logic
```

```
def common_finder(l1, l2):
    output = []
    for i in l1:
        if i in l2:
            output.append(i)
    return output

print(common_finder([1,2,5,8], [1,2,7,6]))
```

Min and max function

```
numbers = [6,60,2]
# print(min(numbers))
# print(max(numbers))
```

Exercise7

Find Greatest Difference

Answer:

```
def greatest_diff(l):
    return max(l) - min(l)
```

```
print(greatest_diff(numbers))
```

Exercise8

```
# find the number of inner List
# [1,2,3, [1,2], [3,4]] , input
# 2
# type()
```

Answer:

```
def sublist_counter(l):
    count = 0
    for i in l:
        if type(i) == list:
            count += 1
    return count
```

```
mixed = [1,2,3, [1,2]]
print(sublist_counter(mixed))
```

Tuples Intro

```
# tuples
# tuples are immutable
# tuple data structure
# tuple can store any data type
# tuples are ordered collections of items
# most important tuples are immutable, once tuple is created you can't update
data inside tuple
# you cannot change(add or delete) values from tuple once it created
# but can add, delete data from list which is present inside tuples
```

```
example = ('one', 'two', 'three')
# no append , no insert , no pop , no remove
# days = ('monday', 'tuesday')
# tuples are faster than lists
```

```
# methods
# count, index
# len function
# slicing
print(example[:2])
```

```
example[0] = 1 #Error
print(example) #Error
```

More about tuples

```
# looping in tuples
# tuple with one element
# tuple without parenthesis
# tuple unpacking
# list inside tuple
# some functions that you can use with tuples

mixed = (1,2,3,4.0)
print(type(mixed))

# for loop and tuple
# for i in mixed:
#     print(i)
# NOTE - You can use while loop too

# tuple with one element
nums = (1,)
words = ('word1',)
# print(type(nums))
# print(type(words))

# tuple without parenthesis
guitars = 'yamaha', 'baton rouge', 'taylor'
# print(type(guitars))

# tuple unpacking
guitarists = ('Maneli Jamal', 'Eddie Van Der Meer', 'Andrew Foy')
guitarist1, guitarist2, guitarist3 = (guitarists)
# print(guitarist1)

# list inside tuples
favorites = ('southern magnolia', ['Tokyo Ghoul Theme', 'landscape'])
favorites[1].pop()
favorites[1].append("we made it ")
# print(favorites)

# min(), max, sum
print(min(mixed))
print(max(mixed))
print(sum(mixed))

# something more about tuples , list , str

# nums = tuple(range(1,11))
print(nums)

nums = list((1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
print(nums)
print(types(nums))

nums = str((1, 2, 3, 4, 5, 6, 7, 8, 9, 10))
print(nums)
print(types(nums))
# ouput = "(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)"

num_list = str([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# print(num_list)
# print(type(num_list))
```

Function returning two values

```
# function returning two values
def func(int1, int2):
    add = int1 + int2
    multiply = int1*int2
    return add, multiply
```

```
# print(func(2,3))
add, multiply = func(2,3)
print(add)
print(multiply)
```

Intro to dictionaries

```
# dictionaries intro
# Q - why we use dictionaries?
# A - Because of limitations of lists , lists are not enough to represent
# real data .

# Example
user = ['Nityam', 24, ['coco', 'kimi no na wa'], ['awakening', 'fairy tale']]
# this list contains user name , age , fav movies , fav tunes
# you can do this but this is not a good way to do this.
```

```
# Q - what are dictionaries
# A - unordered collections of data in key : value pair.
```

```
# how to create dictionaries
user = {'name' : 'Nityam', 'age' : 24}
# print(user)
# print(type(user))
```

```
# second method to create dictionary
user1 = dict(name = 'Nityam', age = 24)
# print(user1)
```

```
# how to access data from dictionary
# NOTE - There is no indexing because of unordered collections of data.
# print(user['name'])
# print(user['age'])
```

```
# which type of data a dictionary can store ?
# anything
# numbers, strings, list , dictionary
```

```
user_info = {
    'name' : 'Nityam',
    'age' : 24,
    'fav_movies' : ['coco', 'kimi no na wa'],
    'fav_tunes' : ['awakening', 'fairy tale'],
}
# print(user_info['fav_movies'])
```

```
# How to add data to empty dictionary
user_info2 = {}
user_info2['name'] = 'Webtech'
user_info2['age'] = 19
```

```

print(user_info2)
# in keyword and iterations in dictionary

user_info = {
    'name' : 'nityam',
    'age' : 24,
    'fav_movies' : ['coco', 'kimi no na wa'],
    'fav_tunes' : ['awakening', 'fairy tale'],
}

# check if key exist in dictionary
# if 'name' in user_info:
#     print('present')
# else:
#     print('not present')

# check if value exist in dictionary ----> values method
# if 24 in user_info.values():
#     print('present')
# else:
#     print('not present')

# if ['coco', 'kimi no na wa'] in user_info.values():
#     print('present')
# else:
#     print('not present')
# values method
# user_info_values = user_info.values()
# print(user_info_values)
# print(type(user_info_values))

# user_info_keys = user_info.keys()
# print(user_info_keys)
# print(type(user_info_keys))

# items method
# user_items = user_info.items()
# print(user_items)
# print(type(user_items))
# [('name', 'nityam'), ('age', 24), ('fav_movies', ['coco', 'kimi no na wa']),
# ('fav_tunes', ['awakening', 'fairy tale'])]

-----
# loops in dictionaries
# for i in user_info:
#     print(i)

# for i in user_info.values():
#     print(i)

# for i in user_info:
#     print(user_info[i])

for i, j in user_info.items():
    print(f"key is {i} and value is {j}")

for i in user_info.items():
    print(i)

```

Add ,Delete Data

```

user_info = {
    'name' : 'nityam',
    'age' : 24,

```

```

    'fav_movies' : ['coco', 'kimi no na wa'],
    'fav_tunes' : ['awakening', 'fairy tale'],
}

# how to add data
# user_info['fav_songs'] = ['song1', 'song2']
# print(user_info)

# pop method
# popped_item = user_info.pop('fav_tunes')
# print(user_info)
# print(popped_item)
# print(type(popped_item)) //string

# pop method
# popped_item = user_info.pop('age')
# print(user_info)
# print(popped_item)
# print(type(popped_item)) //string

# popitem method
# popped_item = user_info.popitem()
# print(user_info)
# print(popped_item)
# print(type(popped_item)) //tuples

```

Update Dictionary

```

user_info = {
    'name' : 'nityam',
    'age' : 24,
    'fav_movies' : ['coco', 'kimi no na wa'],
    'fav_tunes' : ['awakening', 'fairy tale'],
}

more_info = {'name': 'Nityam Webtech', 'State' : 'Gujarat', 'hobbies' :
['coding', 'reading', 'guitar']}

user_info.update(more_info)

more_info = {'name': 'Nityam Webtech', 'State' : 'Gujarat', 'hobbies' :
['coding', 'reading', 'guitar']}

user_info.update(more_info)

user_info.update({})

print(user_info)

```

fromkeys get copy clear method

```

# fromkeys use to assign same values to keys
# d = {'name' : 'unknown', 'age' : 'unknown'}

# d = dict.fromkeys(['name', 'age'], 'unknown')
# print(d)
# d = dict.fromkeys(('name', 'age'), 'unknown')
# print(d)
# d = dict.fromkeys("xyz", 'unknown')
# print(d)
# d = dict.fromkeys(range(1,11), 'unknown')
# print(d)
# d = dict.fromkeys(range(1,11), ['unknown', 'unknown']) //Meaning Less

```

```

# print(d)

# get method (useful)
d = {'name' : 'Nityam', 'age' : 'unknown'}
# print(d['name'])
# print(d['names'])

# print(d.get('name'))
# print(d.get('names')) better

# if 'name' in d:
#     print('present')
# else:
#     print('not present')

# if d.get('name'):
#     print('present')
# else:
#     print('not present')

# if None ----> False,     else -----> True

# d.clear()
# print(d)

# d1 = d.copy()
# print(d1.popitem())
# print(d)

# d1 = d
# print(d1.popitem())
# print(d)

# print(d1 is d)
# print(d1 == d)

# more about get, two same keys in dictionaries
user = {'name':'Nityam', 'age':34, 'age':24}
# print(user.get('fav', 'not found ! '))
print(user)

```

Exercise

```

# exercise
# define a function that takes a number(n)
# return a dictionary containing cube of numbers from 1 to n

# example
# cube_finder(3)
# {1:1, 2:8, 3:27}

```

Answer

```

# cube finder
def cube_finder(n):
    cubes = {}
    for i in range(1,n+1):
        cubes[i] = i**3
    return cubes

print(cube_finder(10))

```

Exercise

Make Letter Counter take string as a Input ,output should be in Dictionary format

Answer

```

def word_counter(s):
    count = {}
    for char in s:
        count[char] = s.count(char)
    return count

```

```
print(word_counter('webtech'))
```

Exercise

Take Dictionary as input from user, dynamically and print properly each key value pair in separate line.

Answer

```
user = {}

name = input('What is your name : ')
age = input('what is your age : ')
fav_movies = input('your fav movies separated by comma ').split(',')
fav_songs = input('your fav songs separated by comma ').split(',')

user['name'] = name
user['age'] = age
user['fav_movies'] = fav_movies
user['fav_songs'] = fav_songs
#print(user)

for key, value in user.items():
    print(f"{key} : {value}")
=====General Exercise=====
```

Exercise

Python program to find the factorial value of natural numbers up to n using recursive function

Answer

```
def recur_sum(n):
    """Function to return the sum
    of natural numbers using recursion"""
    if n <= 1:
        return n
    else:
        return n * recur_sum(n-1)

# change this value for a different result
num = 3

# uncomment to take input from the user
#num = int(input("Enter a number: "))

if num < 0:
    print("Enter a positive number")
else:
    print("The sum is",recur_sum(num))
```

Exercise

```
# Python Program to find the factors of a number

# define a function
def print_factors(x):
    # This function takes a number and prints the factors

    print("The factors of",x,"are:")
    for i in range(1, x + 1):
        if x % i == 0:
            print(i)

# change this value for a different result.
num = 320

# uncomment the following line to take input from the user
#num = int(input("Enter a number: "))

print_factors(num)
```

SET

```
# set data type
# unordered collection of unique items
```

```
# s = {1,2,3,2}
print(s)
print(s[1]) #Error
```

```
-----
s = {1,1.1, 2.3, 'string'}
```

```
# print(s)
-----
```

```
# l = [1,2,3,4,5,5,5,5,6,7,7,8]
# remove duplicate
s2 = set(l)
print(s2)
```

```
# l = [1,2,3,4,5,5,5,5,6,7,7,8]
# remove duplicate
# s2 = list(set(l))
# print(s2)
-----
```

```
# s.add(4)
print(s)
```

```
# s.add(5)
# s.add(4)
print(s)
```

```
# s.remove(3)
print(s)
```

```
# s.discard(5)
print(s)
```

```
#s.clear()
print(s)
```

```
# s1 = s.copy()
# print(s1)
print(s)
-----
```

```
# in keyword in sets and for loop
```

```
s = {'a', 'b', 'c'}
```

```
# in keyword to check if item is present or not in set
if 'a' in s:
    print("present")
else:
    print("not present")
```

```
# for loop
for item in s:
    print(item)
```

```
s1 = {1,2,3,4}
s2 = {3,4,5,6}
```

```
# union
```

```
# intersection
```

```
# {1,2,3,4,5,6}
```

```
# |
# union_set = s1 | s2
# print(union_set)
print(s1 & s2)

=====
=====
```

list comprehension

```
# list comprehension
# with the help of list comprehension we can create of list in one line
```

```
# create a list of squares from 1 to 10
# squares = []
# for i in range(1,11):
#     squares.append(i**2)
# print(squares)
```

```
# square2 = [i**2 for i in range(1,11)]
# print(square2)
```

```
# negative = []
# for i in range(1,11):
#     negative.append(-i)
# print(negative)

# new_negative = [-i for i in range(1,11)]
```

```
names = ['Vaibhav', 'Akash', 'Kishan']
# new_list = ['V', 'A', 'K']
# new_list = []
# for name in names:
#     new_list.append(name[0])
# print(new_list)
```

```
new_list2 = [name[0] for name in names]
print(new_list2)
```

Exercise:

```
# define a function that take list of strings
# list containing reverse of every string
```

```
# NOTE - USE LIST COMPREHENSION because we already did this exercise
# using normal method
# example
# l = ['abc', 'tuv', 'xyz']
# reverse_string(l) ----> ['bac', 'vut', 'zyx']
```

Answer:

```
def reverse_strings(l):
    return [name[::-1] for name in l]

print(reverse_strings(['abc', 'tuv', 'xyz']))
```

```
def reverse_str(l):
    new_list = []
    for name in l:
        new_list.append(name[::-1])
    return new_list

print(reverse_str(['abc', 'tuv', 'xyz']))
```

Exercise:

Create list comprehension which will print only even numbers from givent range.

Answer:

```
# list comprehension with if statement
```

```

numbers = list(range(1,11))
# [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
# even_nums = [2,4,6]
nums = []
for i in numbers:
    if i%2 == 0:
        nums.append(i)
print(nums)

even_nums = [i for i in numbers if i%2 == 0]
even_num2 = [i for i in range(1,11) if i%2 == 0]
print(even_nums)
print(even_num2)
odd_nums = [i for i in range(1,11) if i%2 != 0]
print(odd_nums)

```

Exercise:

```

# num to string
# define a function

# example
# input ----> [True, False , [1,2,3], 1, 1.0, 3]
# output -----> ['1', '1.0', '3']

```

Answer:

```

def num_to_string(l):
    return [str(i) for i in l if (type(i) == int or type(i) == float) ]

print(num_to_string([True, False , [1,2,3], 1, 1.0, 3]))

```

Exercise:

```

# list comprehension with if else

nums = [1,2,3,4,5,6,7,8,9,10]
# new_list = [-1,4, -3, 8 ]

```

Answer:

```

nums = [1,2,3,4,5,6,7,8,9,10]

new_list = []
for i in nums:
    if i%2 == 0:
        new_list.append(i*2)
    else:
        new_list.append(-i)
print(new_list)

new_list2 = [i*2 if (i%2 == 0) else -i for i in nums]
print(new_list2)

```

Nested list comprehension

```

# list comprehension in nested list

example = [[1,2,3], [1,2,3], [1,2,3]]
nested_comp = [[i for i in range(1,4)] for j in range(3) ]
print(nested_comp)

new_list = []
for j in range(3):
    new_list.append([1,2,3])

```

```

=====

```

Dictionary Comprehension

```
# dictionary comprehension
# square = {1:1, 2:4, 3:9}
square = {num:num**2 for num in range(1,11) }

-----

square = {f"Square of {num} is":num**2 for num in range(1,11) }

-----

for k,v in square.items():
    print(f"{k} : {v}")

-----

string = "Nityam"
# for i in string:
#     print(i)
# {'N' : 1 , 'a' : 1}
word_count = {char:string.count(char) for char in string}
print(word_count)
```

Dictionary comprehension with if else

Exercise:

```
# dictionary comprehension with if else
# d = {1:'odd' , 2:'even'}
```

Answer:

```
# dictionary comprehension with if else
# d = {1:'odd' , 2:'even'}
odd_even = {i:('even' if i%2==0 else 'odd') for i in range(1,11) }
print(odd_even)
```

Sets comprehension

```
# sets comprehension
s = {k**3 for k in range(1,11)}
print(s)
```

```
names = ['Jiten', 'mohit', 'rohit']
first = {name[0] for name in names}
print(first)
```

Intro to * args

```
# *operator
# *args

def total(a,b):
    return a+b
print(total(3,4,5))

def all_total(*args):
    print(args)
    print(type(args))
print(all_total(3,4,5))

-----
-----

all_total(1,2,3,4)

def all_total(*args):
    # (1,2,3,4,5,65)
    total = 0
```

```

    for num in args:
        total += num
    return total

print(all_total(1,2,3,4))

```

Exercise : make sum of only even num

* args with normal parameter

*args with normal parameter

```

def multiply_nums(*args):
    multiply = 1

    for i in args:
        multiply *= i
    return multiply

print(multiply_nums(2,4,3,4))

```

```

def multiply_nums(num,*args):
    multiply = 1
    print(num)
    print(args)

    for i in args:
        multiply *= i
    return multiply

print(multiply_nums(2,4,3,4))

```

Args as argument

```

def multiply_nums(*args):
    multiply = 1
    print(args) # ([2,3,4])
    for i in args:
        multiply *= i
    return multiply

nums = (2,3,4)
nums = [2,3,4]
print(multiply_nums(*nums)) # unpack

```

Exercise:

```

# define a function
# input

# num , iterable(tuple , list)containing numbers as args

# example
# nums = [1,2,3]
# to_power(3,*nums)

# output
# list ---> [1, 8, 27]

# if user didn't pass any args then give a user a message 'hey you didn't pass
# args

# else

```

```
# return list

# NOTE - USE list comprehension
Answer:

def to_power(num, *args):
    if args:
        return [i**num for i in args]
    else:
        return "You didn't pass any args"

nums = [1,2,3]
print(to_power(3, *nums))
```

**** Kwargs(Keyword Arguments)**

```
# kwargs (keyword arguments)
# **kwargs (double star operator)

# kwargs as a parameter
def func(**kwargs):
    print(kwargs)
    print(type(kwargs))
func(fname='Nityam',lname='webtech')

-----
# kwargs (keyword arguments)
# **kwargs (double star operator)

# kwargs as a parameter
def func(**kwargs):
    for k, v in kwargs.items():
        print(f"{k} : {v}")

func(fname='Nityam',lname='webtech')

-----
# kwargs (keyword arguments)
# **kwargs (double star operator)

# kwargs as a parameter
def func(name,**kwargs):
    for k, v in kwargs.items():
        print(f"{k} : {v}")

func('rajkot',fname='Nityam',lname='webtech')

-----
# kwargs (keyword arguments)
# **kwargs (double star operator)

# kwargs as a parameter
def func(**kwargs):
    for k, v in kwargs.items():
        print(f"{k} : {v}")

# dictionary unpacking
d= {'name':'Nityam', 'age' : 24}
func(**d)
```

Function with all type of parameters

```
# functions with all parameters
# very important to understand

# PADK

# parameters
```

```
# *args
# default parameters
# **kwargs

def func(name, *args, last_name = 'unknown', **kwargs):
    print(name)
    print(args)
    print(last_name)
    print(kwargs)

func('Nityam', 1,2,3, a = 1, b = 2)
```

Exercise:

```
# def a function
names = ['nityam', 'webtech']

# print(func(names))
# print(func(names, reverse_str = True))
```

Answer:

```
# function
# list , reverse_str = True
# list

def func(l, **kwargs):
    if kwargs.get('reverse_str') == True:
        return [name[::-1].title() for name in l]
    else:
        return [name.title() for name in l]
```

```
names = ['nityam', 'webtech']
print(func(names, reverse_str = True))
```

```
=====
=====
```

Lambda Expression

```
# lambda expressions (anonymous function)
```

```
def add(a,b):
    return a+b

add2 = lambda a,b : a+b
print(add2(2,3))

multiply = lambda a,b : a*b
print(multiply(2,3))

# print(add)
# print(add2)
# print(multiply)
-----
# lambda expression practice
# def is_even(a):
#     return a %2 == 0 # a%2 == 0 ----> true , false

# print(is_even(5))
# iseven2 = lambda a : a%2==0
# print(iseven2(6))
# def last_char(s):
#     return s[-1]

# last_char = lambda s : s[-1]
# print(last_char('nityam'))

# lambda with if , else
def func(s):
    if len(s) > 5:
        return True
```

```

return False

def func(s):
    return len(s) > 5

func = lambda s : True if len(s) > 5 else False
print(func('abcdefg'))

func = lambda s : len(s) > 5
print(func('abcdefg'))

```

Enumerate function

We use enumerate function with for loop to track position of our

item in iterable

```

# How we can do this without enumerate function
names = ['abc', 'abcdef', 'nityam']
# 0 -- 'abc'
# 1 -- 'abcdef'
# pos = 0
# for name in names:
#     print(f"{pos} ----> {name}")
#     pos += 1

```

with enumerate function

```

# for pos, name in enumerate(names):
#     print(f"{pos} ----> {name}")

```

Define a function that take two arguments

1.) list containing string

2.) string that want to find in your list

and this function will return the index of string in your list and

if string is not present then return -1

```

def find_pos(l, target):
    for pos, name in enumerate(l):
        if name == target:
            return pos
    return -1

```

```

names = ['abc', 'abcdef', 'nityam']
print(find_pos(names, 'nityam'))

```

Map function

map function

Returns a list of the results after applying the given function
to each item of a given iterable (list, tuple etc.)

```

numbers = [1,2,3,4]

```

```

def square(a):
    return a**2

```

```

#print(map(square, numbers))

```

```
squares = list(map(square, numbers))
print(squares)

squares = list(map(lambda a:a**2, numbers))
print(squares)
# list comp
```

```
squares_new = [i**2 for i in numbers]
print(squares_new)
-----
def square(a):
    return a**2
```

```
new_list = []
for num in numbers:
    new_list.append(square(num))

print(new_list)
-----
```

Exercise : Find length of individual and make list

```
names = ['abc', 'abcd', 'abcde']

length = list(map(len, names))

print(length)
-----
names = ['abc', 'abcd', 'abcde']
length = map(len, names)

for i in length:
    print(i)
-----
# List of strings
# map() can listify the list of strings individually

l = ['sat', 'bat', 'cat', 'mat']

test = list(map(list, l))
print(test)
-----
```

Exercise: # Add two lists using map and lambda

```
numbers1 = [1, 2, 3] numbers2 = [4, 5, 6]
```

Answer:

```
result = map(lambda x, y: x + y, numbers1, numbers2)

print(list(result))
```

Filter Function

```
# filter function

def is_even(a):
    return a%2 == 0

numbers = [3,4,2,1,5,6,9,8,10]

evens = tuple(filter(is_even, numbers))
print(evens)
```

```
evens = tuple(filter(lambda a:a%2==0, numbers))
print(evens)
```

```
evens = filter(lambda a:a%2==0, numbers)
for i in evens:
    print(i)
```

```
new_evens = [i for i in numbers if i%2==0]
```

```
print(evens)
print(list(evens))
print(new_evens)
```

```
# iterator vs iterables
```

```
numbers = [1,2,3,4] # ,list, tuples, string --- iterables
```

```
for a in numbers:
    print(a)
```

```
for b in numbers:
    print(b)
```

How Loop Works in iterables

- 1) Call iter function
- 2) iter(numbers)----->iterators
- 3) next(iter(numbers)) or next(iterators)

Ex:

```
number_iter = iter(numbers)
print(next(number_iter))
print(next(number_iter))
print(next(number_iter))
print(next(number_iter))
check----->print(iter(numbers))
```

Iterable objects is a generalization of arrays. That's a concept that allows to make any object useable in a `for..of` loop.

Arrays by themselves are iterable. But not only arrays. Strings are iterable too, and many other built-in objects as well.

```
squares = map(lambda a:a**2, numbers) # iterator
print(squares)
for i in squares:
    print(i)
for j in squares:
    print(j)
```

```
print(next(squares))
print(next(squares))
print(next(squares))
```

```
print(next(numbers)) # Error
```

Zip function

```
# zip function
user_id = ['user1', 'user2', 'user3']
names = ['jay', 'mohit', 'rohit']
```

```
# ('user1', 'jay'), ('user2', 'mohit')
```

```
print (zip(user_id, names)) # Return Zip object ,it's iterator
```

```
print(list(zip(user_id, names)))
```

Tuples to dict

```
# example = [('a',1), ('b', 2)]
# print(dict(example))
```

```
print(dict(zip(user_id, names)))
```

3 List Example:

```
# zip function
user_id = ['user1', 'user2', 'user3']
names = ['jay', 'mohit', 'rohit']
last_names = ['patel', 'shah', 'sharma']
```

```
print(list(zip(user_id, names, last_names)))
print(dict(zip(user_id, names, last_names))) #Error .....
```

Exercise:

```
l = [(1,2), (3,4), (5,6), (7,8)] Required output-->l1=[1,3,5,7] l2=[2,4,6,8]
```

Answer:

```
#print(list(zip(*l)))
```

```
l1, l2 = (list(zip(*l)))
```

```
print(list(l1))
print(list(l2))
```

Exercise:

```
l1=[1,3,5,7] l2=[2,4,6,8] Required output-->new_list=[list of greater number form two list]
```

Answer:

```
for pair in zip(l1,l2):
    new_list.append(max(pair))
```

```
print(new_list)
```

Exercise:

```
# define a function take any no of lists containing number
# [1,2,3] , [4,5,6], [7,8,9]
# return average
# (1+4+7)/3 , (2,5,8)/3 , (3,6,9)/3
```

```
# try to make this anonymous function in one line using lambda
```

Answer:

```
# def average_finder(*args):
#     average = []
#     for pair in zip(*args):
#         average.append(sum(pair)/len(pair))
#     return average
```

```
average_finder = lambda *args: [sum(pair)/len(pair) for pair in zip(*args)]
```

```
print(average_finder([1,2,3] , [4,5,6], [7,8,9]))
```

any and all function

```
# any , all function
```

```
numbers1 = [13,1,9,7,10]
numbers2 = [1,2,3,4,5,6]
```

```

# evens1 = []
# for num in numbers1:
#     evens1.append(num%2==0)

#print(evens1)

# print(all([True, True, True, True, True])) # ---> #True
# print(all([True, False, True, True, True])) # ---> #False

# print(any([num%2==0 for num in numbers1]))
#print(all([num%2==0 for num in numbers1]))
Exercise : Make sum of (1,2,3,4,8.9) if there is string
(1,2,3,4,8.9,Nityam,[1,2,4,5])then print wrong input
# any all function
Answer:
def my_sum(*args):
    # args = ()
    if all([(type(arg) == int or type(arg) == float) for arg in args]):
        total = 0
        for num in args:
            total += num
        return total
    else:
        return "Wrong input"

print(my_sum(1,2,3,4,8.9,Nityam,[1,2,4,5]))

```

Advance min() and max()

```

# advance min() and max()

# numbers = [1,2,4,5,7]
# print(min(numbers))
# print(max(numbers))

def func(item):
    return len(item)

names = ['nityam','jay','ab']
print(max(names, key = func))
print(min(names, key = func))

names = ['nityam','jay','ab']

print(max(names, key = lambda item : len(item)))

students = {
    'jay' : {'score':50, 'age':24},
    'mohit' : {'score':75, 'age':19},
    'rohit' : {'score':76, 'age':23}
}
print(max(students, key = lambda item : students[item]['age']))
print(max(students, key = lambda item : students[item]['score']))

-----
# students2 = [
#     {'name':'jay', 'score':90, 'age':24},
#     {'name':'mohit', 'score':70, 'age':19},
#     {'name':'rohit', 'score':60, 'age':25}
# ]

# print(max(students2, key = lambda item:item.get('score')))
# print(max(students2, key = lambda item:item.get('score'))['name'])
# print(max(students2, key= lambda item:item.get('age'))['name'])

```

Advance sorted function

```
fruits = ['grapes', 'mango', 'apple']
# sort
# fruits.sort()
# print(fruits)
```

```
fruits2 = ('grapes', 'mango', 'apple')
# fruits2.sort()
# print(fruits2) # Error
# print(sorted(fruits2)) # Emutable
```

```
fruits3 = {'grapes', 'mango', 'apple'}
# print(sorted(fruits3))
```

```
guitars = [
    {'model': 'yamaha f310', 'price':8400},
    {'model': 'faith naptune', 'price':50000 },
    {'model': 'faith apollo venus', 'price': 35000},
    {'model': 'taylor 814ce', 'price': 450000}
]
print(sorted(guitars, key = lambda d:d['price'])) # also can use .get method-revise dict
chapter
```

```
print(sorted(guitars, key = lambda d:d['price'] , reverse = True))
sorted_guitars = sorted(guitars, key = lambda d:d['price'], reverse = True)
print(sorted_guitars)
```

More about functions

```
# what are doc strings
# how to write docstring
# how to see doctstrings
# what is help function
```

```
def add(a,b):
    ''' this function takes 2 numbers and return their sum '''
    return a+b
```

```
# print(add.__doc__)
```

```
# len , sum , max , min , sorted
# print(len.__doc__)
```

```
# print(sum.__doc__)
# sum
# print(help(sum))
```

Preface Of Decorators

```
# You have to have a complete understanding of functions,
# first class function / closures
# about decorators
```

```
def square(a):
    return a**2
```

```
s = square
# print(s(7))
```

```

print(s.__name__)
print(square.__name__)
print(s)
print(square)

```

Pass function as argument

```

# map
# def square(a):
#     return a**2
l = [1,2,3,4]
print(list(map(square,l)))
# print(list(map(lambda a : a**2, l)))
.....

```

Exercise: Make UDF act as a map

Ans:

```

def my_map(func, l):
    new_list = []
    for item in l:
        new_list.append(func(item))
    return new_list

print(my_map(square,l))

print(my_map(lambda a : a**3, l))

.....
def my_map2(func, l):
    return [func(item) for item in l]

print(my_map2(lambda a : a**3, l))

```

Function returning function(Closure)/First class Function

```

# function returning function

def outer_func():
    def inner_func():
        print('inside inner func')
    return inner_func

var = outer_func()
var()

or

def outer_func():
    def inner_func():
        print('inside inner func')
    return inner_func()

var = outer_func()

.....
def outer_func2(msg):
    def inner_func2():
        print(f"message is {msg}")
    return inner_func2

var = outer_func2("hello there ! ")
var()

```

```

# function returning function (closure) practice

```

```
def to_power(x): # x = 3
    def calc_power(n): # n = 2
        return n**x
    return calc_power
```

```
cube = to_power(3)
print(cube(2))
```

```
square = to_power(2)
print(square(4))
```

Decorators Intro

Decorators is a function - enhance the functionality of other functions

```
def decorator_function(any_function):
    def wrapper_function():
        print('this is awesome function')
        any_function()
    return wrapper_function
```

this is awesome function

```
def func1():
    print('this is function 1 ')
```

this is awesome function

```
def func2():
    print('this is function 2')
```

```
func1 = decorator_function(func1)
func1()
```

Decorators is a function - enhance the functionality of other functions
@ use for decorator

```
def decorator_function(any_function):
    def wrapper_function():
        print('this is awesome function')
        any_function()
    return wrapper_function
```

this is awesome function

```
@decorator_function # shortcut
def func1():
    print('this is function 1 ')
```

```
func1()
```

```
@decorator_function
def func2():
    print('this is function 2')
```

```
func2()
```

```
# func1 = decorator_function(func1)
# func1()
```

Decorators Part 2

Single Argument:

```
def decorator_function(any_function):
    def wrapper_function(*args, **kwargs):
        print('this is awesome function')
        any_function(*args, **kwargs)
    return wrapper_function
@decorator_function
def func(a):
    print(f'this is function with argument {a}')

func(5)
```

```
def decorator_function(any_function):
    def wrapper_function(*args, **kwargs):
        print('this is awesome function')
        return any_function(*args, **kwargs)
    return wrapper_function
```

```
@decorator_function
```

```
def add(a,b):
    return a + b
```

```
print(add(2,3))
```

```
# add = decorator_function(add(a,b))
# add()
```

Decorators part 3

```
#from functools import wraps
def decorator_function(any_function):
    @wraps(any_function)
    def wrapper_function(*args, **kwargs):
        """ this is wrapper function """
        print('this is awesome function')
        return any_function(*args, **kwargs)
    return wrapper_function
```

```
@decorator_function
```

```
def add(a,b):
    ''' this is add function '''
    return a + b
```

```
print(add.__doc__)
print(add.__name__)
```

Exercise:

```
# @print_function_data
def add(a,b):
    '''This function takes two numbers as arguments and return their sum '''
    return a+b

print(add(4,5))
# output
# you are calling add function
# This function takes two numbers as parameters and return their sum
# 9
```

Answer:

```
from functools import wraps
```

```
def print_function_data(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print(f"You are calling {function.__name__} function")
        print(f"{function.__doc__}")
        return function(*args, **kwargs)
    return wrapper

@print_function_data
def add(a,b):
    '''This function takes two numbers as arguments and return their sum '''
    return a+b

print(add(4,5))
# output
# you are calling add function
# This function takes two numbers as parameters and return their sum
# 9
```

```
import time
```

```
t1 = time.time()
print('this is line one ')
x = 5
if x == 5 :
    print('x is equal to 5')
print('this is line two')
print('this is line two')
print('this is line two')
print('this is line two')
print('this is line two')
t2 = time.time()
print(t2-t1)
```

Exercise:

```
# @calculate_time
# def func():
#     print('this is function')

# func()
# # this function took 3 sec to run
```

Answer:

```
from functools import wraps
import time
def calculate_time(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print(f'Executing .... {function.__name__}')
        t1 = time.time()
        returned_value = function(*args, **kwargs)
        t2 = time.time()
        total_time = t2 - t1
        print(f'This function took {total_time} seconds')
        return returned_value
```

```

return wrapper

@calculate_time
def square_finder(n):
    return [i**2 for i in range(1,n+1)]

square_finder(1000)

```

Exercise:

```

from functools import wraps
def only_int_allow(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        if all([type(arg) == int for arg in args]):
            return function(*args, **kwargs)
        print("Invalid arguments")

        # data_types = []
        # for arg in args:
        #     data_types.append(type(arg)==int)
        # if all(data_types):
        #     return function(*args, **kwargs)
        # else:
        #     print("Invalid arguments")
    return wrapper

@only_int_allow
def add_all(*args):
    total = 0
    for i in args:
        total += i
    return total
print(add_all(1,2,3,4,5, 'nityam'))

```

Decorators with arguments

```

from functools import wraps
def only_data_type_allow(data_type):
    def decorator(function):
        @wraps(function)
        def wrapper(*args, **kwargs):
            if all([type(arg) == data_type for arg in args]):
                return function(*args, **kwargs)
            print("Invalid arguments")
        return wrapper
    return decorator

@only_data_type_allow(str)
def string_join(*args):
    string = ''
    #string=0
    for i in args:
        string += i
    return string

print(string_join('nityam', 'webtech', 'a', 8))

```

Intro to generators

```

# generators
# generators are iterators

# iterator , iterable

l = [1,2,3] # iterable

# for i in l:
#     print(i)

# i = iter(l)
# print(next(i))
# print(next(i))
# print(next(i))
# print(next(i))
# next(l)


# for num in map(lambda a :a**2, l):
#     print(num)

# l = [1,4,9,16]
# memory ---- [.....] , list
# memory ---- (9)
# create your first generator with generator function
# 1.) generator function

# 10 , 1 to 10

Normal Condition:
def nums(n):
    for i in range(1,n+1):
        print(i)
nums(10)
-----
def nums(n):
    for i in range(1,n+1):
        yield(i)
nums(10)
print(nums(10))
-----
def nums(n):
    for i in range(1,n+1):
        yield
nums(10)
print(nums(10))
-----
def nums(n):
    for i in range(1,n+1):
        yield(i)
for number in nums(10):

    print(number)

-----

numbers = nums(10)

for nums in nummbbers:

    print(number)

for nums in nummbbers:

    print(number)

```

Exercise:

```
# define generator function
# take one number as argument
# generate a sequence of even numbers from 1 to that number
# 5 --> 2,4
# 7 --> 2,4,6
```

ANSWER:

```
def even_generator(n):
    for num in range(1,n+1):
        if num % 2 == 0:
            yield(num)
def even_generator(n):
    for num in range(2,n+1,2):
        yield(num)
for num in even_generator(20): # why Its Print Multiple times???
    print(num)

even_nums = even_generator(20)
for num in even_nums:
    print(num)
```

=====

Generator Comprehension

```
square = [i**2 for i in range(1,11)]
print(square)

square = (i**2 for i in range(1,11))

for num in square:
    print(num)

print(next(square))
print(next(square))
print(next(square))
```

=====

List vs Generators

```
import time
# list vs generator
# memory usage , time
# when to use list , when to use generator
Check Memory
# l = [i**2 for i in range(10000000)] # 10 million

# g = (i**2 for i in range(10000000)) # 10 million

Now check time
# t1 = time.time()
# l = [i**2 for i in range(10000000)] # 10 million
# print(time.time() - t1)

t1 = time.time()
g = (i**2 for i in range(10000000)) # 10 million
print(time.time() - t1)
```

=====

=====

Object Oriented Programming Intro

```
# OBJECT ORIENTED PROGRAMMING
```

```
# COMMON TOPIC IN ALMOST ALL POPULAR PROGRAMMING LANGUAGES (PYTHON, C++, JAVA)
# WITH COMMON IDEA BUT WITH DIFFERENT SYNTAX

# OOP IS JUST A STYLE/WAY TO WRITE A CODE

# VERY HELPFUL IN CREATING REAL WORLD PROGRAMS

# WE WILL SEE OTHER ADVANTAGES WHEN WE WILL START LEARNING OOP

# class , object(instance), method

# list class
# list object
# method

l = [1,2,3,4]
l2 = [4,5,6,7]
l3 = ['nityam', 'webtech']
l.append(8)
```

OOP Create Your First Class

```
# OBJECTIVES
# WHAT IS CLASS
# HOW TO CREATE AN CLASS
# WHAT IS INIT METHOD , constructor
# WHAT ARE ATTRIBUTES, INSTANCE VARIABLES
# HOW TO CREATE OUR OBJECT
```

```
class Person:
    def __init__(self, first_name, last_name, age):
        # instace variables
        print('init method  called')
        self.first_name = first_name
        self.last_name = last_name
        self.age = age
```

```
p1 = Person('Nityam', 'Webtech', 24)
p2 = Person('Job', 'Heaven', 19)
```

```
print(p1.first_name)
print(p2.first_name)
```

```
#print(p1.person_first_name)
#print(p2.person_first_name)
```

Excercise:

```
# Create a laptop class with attributes like brand name , model name , price
# create two objects/instance of your laptop class
```

Answer:

```
class Laptop:
    def __init__(self, brand, model_name, price):
        #instance variables
        self.brand = brand
        self.name = model_name
        self.price = price
        self.laptop_name = brand + ' ' + model_name
```

```
laptop1 = Laptop('hp', 'au114tx', 63000)
```

```
Exmple:car1 = car('wagonr','hatchback',250000,'petrol')
```

OOP Instance Methods

```
# instance methods
class Person:
    def __init__(self,first_name, last_name, age):
        self.first_name = first_name
        self.last_name = last_name
        self.age = age

    def full_name(self):
        return f"{self.first_name} {self.last_name}"
```

```
p1 = Person('raj', 'patel', 10)
p2 = Person('paras', 'shah', 24)
# print(p2.full_name())
print(person.full_name(p2))
```

```
l = [1,2,3]
# clear , pop
# l.clear()
# list.clear(l)
# print(l)
# l.append(10)
# list.append(l,10)
print(l)
```

```
class Person:
    def __init__(self,first_name, last_name, age):
        self.first_name = first_name
        self.last_name = last_name
        self.age = age

    def full_name(self):
        return f"{self.first_name} {self.last_name}"

    def is_above_18(self):
        return self.age>18
```

```
p1 = Person('raj', 'patel', 10)
p2 = Person('paras', 'shah', 24)
print(p1.is_above_18())
```

Exercise:laptop1 = Laptop('hp', 'au114tx', 63000) make function apply_discount() and apply dicount 5% of price

Answer:

```
class Laptop:
    def __init__(self, brand, model_name, price):
        #instance variables
        self.brand = brand
        self.name = model_name
        self.price = price
        self.laptop_name = brand + ' ' + model_name

    def apply_discount(self,num):
        # self.price
```

```

    disc = (num/100)*self.price
    return self.price - disc

```

```

laptop1 = Laptop('hp', 'au114tx', 63000)
laptop2 = Laptop('apple', 'macbook pro', 230000)
print(laptop1.apply_discount(10))

```

Class Variable

```

# class variable
# example 1
class Circle:
    pi = 3.14
    def __init__(self, radius):
        self.radius = radius
    def calc_circumference(self):
        return 2*Circle.pi*self.radius

c = Circle(4)
c1 = Circle(5)
# print(c.calc_circumference())

# Example 2
class Laptop:
    discount_percent = 10
    def __init__(self, brand, model_name, price):
        #instance variables
        self.brand = brand
        self.name = model_name
        self.price = price
        self.laptop_name = brand + ' ' + model_name

    def apply_discount(self):
        # self.price
        off_price = (Laptop.discount_percent/100)*self.price
        return self.price - off_price

laptop1 = Laptop('hp', 'au114tx', 63000)
laptop2 = Laptop('apple', 'macbook pro', 230000)
# print(laptop1.apply_discount())

```

Class Variable part 2

```

# change class variable
# how to print name space for objects
# What if we use self instead of class name for class variables
#

class Laptop:
    discount_percent = 10
    def __init__(self, brand, model_name, price):
        #instance variables
        self.brand = brand
        self.name = model_name
        self.price = price
        self.laptop_name = brand + ' ' + model_name

    def apply_discount(self):
        # self.price
        off_price = (self.discount_percent/100)*self.price
        return self.price - off_price

```

```
laptop1 = Laptop('hp', 'au114tx', 63000)
laptop2 = Laptop('apple', 'macbook pro', 230000)
laptop2.discount_percent = 50
print(laptop2.__dict__)
print(laptop2.apply_discount())
```

OOP Class Methods

```
# class methods
# difference between class methods and instance methods

class Person:
    count_instance = 0 # class variable / class attribute
    def __init__(self, first_name, last_name, age):
        Person.count_instance += 1
        self.first_name = first_name
        self.last_name = last_name
        self.age = age

    @classmethod
    def count_instances(cls):
        return f"You have created {cls.count_instance} instances of {cls.__name__} class"

    def full_name(self):
        return f"{self.first_name} {self.last_name}"

    def is_above_18(self):
        return self.age > 18

p1 = Person('Raj', 'Shah', 10)
p2 = Person('Raj', 'Shah', 24)
print(Person.count_instances())
```

Class Method as a constructor

```
# class method as a constructor
class Person:
    count_instance = 0 # class variable / class attribute

    def __init__(self, first_name, last_name, age):
        Person.count_instance += 1
        self.first_name = first_name
        self.last_name = last_name
        self.age = age

    @classmethod
    def from_string(cls, string):
        first, last, age = string.split(',')
        return cls(first, last, age)

    @classmethod
    def count_instances(cls):
        return f"You have created {cls.count_instance} instances of {cls.__name__} class"

    def full_name(self):
        return f"{self.first_name} {self.last_name}"

p1 = Person('Nityam', 'Webtech', 10)
p2 = Person('Raj', 'Patel', 24)

# p3 = Person('Jay,Shah,24')

p3 = Person.from_string('Jay,Shah,24')
print(p3.full_name())
```

OOP Static Method

```
class Person:
    count_instance = 0 # class variable / class attribute

    def __init__(self, first_name, last_name, age):
        Person.count_instance += 1
        self.first_name = first_name
        self.last_name = last_name
        self.age = age

    @classmethod
    def from_string(cls, string):
        first, last, age = string.split(',')
        return cls(first, last, age)

    @classmethod
    def count_instances(cls):
        return f"You have created {cls.count_instance} instances of {cls.__name__} class"

    @staticmethod
    def hello():
        print('hello, static method called')

    def full_name(self):
        return f"{self.first_name} {self.last_name}"

    def is_above_18(self):
        return self.age > 18
```

```
p1 = Person('Nityam', 'Webtech', 10)
p2 = Person('Raj', 'Patel', 24)

p3 = Person.from_string('Jay,Shah,24')
Person.hello()
```

Exercise:

```
phone1 = Phone('Nokia', '1100', -1000)
print(phone1.brand)
print(phone1.model_name)
print(phone1.price)
print(phone1.complete_specification)
```

Answer:

```
class Phone:
    def __init__(self, brand,model_name, price):
        self.brand = brand
        self.model_name = model_name
        self.price = max(price,0)
        #if price > 0:
            #self._price = price
        #else
            # self._price = 0
        self.complete_specification = f"{self.brand} {self.model_name} and price is {self._price}"

phone1 = Phone('Nokia', '1100', -1000)
print(phone1.brand)
print(phone1.model_name)
print(phone1.price)
print(phone1.complete_specification)
```

Encapsulation, Abstraction, Naming Convention, Name Mangling

```
# Encapsulation
# Abstraction
# Some special naming convention
# Name Mangling , __name (not a convention)

class Phone:
    def __init__(self, brand,model_name, price):
        self.brand = brand
        self.model_name = model_name
        self._price = price
        # self.__price = price
        self.complete_specification = f"{self.brand} {self.model_name} and price is {self._price}"

    def make_a_call(self, phone_number):
        print(f"calling {phone_number} ...")

    def full_name(self):
        return f"{self.brand} {self.model_name}"

    def send_mesaage(self):
        pass # twilio

phone1 = Phone('nokia', '1100',1000)
# print(phone1._price)

# phone1.__price = -1000
# print(phone1.__price)

# print(phone1.__price) #Error

#print(phone1.__dict__)
print(phone1._Phone__price)
phone1._Phone__price = -1000
print(phone1._Phone__price)

# _name # convention of private name
# __name__ # dunder/magic methods

# l = [3,4,1,2]
# l.sort() # tim sort , Timsort is a hybrid stable sorting algorithm, derived from merge sort and insertion sort,
designed to perform well on many kinds of real-world data.
# print(l)
```

Inheritance Intro

```
class Phone: # base class / parent class
    def __init__(self, brand,model_name, price):
        self.brand = brand
        self.model_name = model_name
        self._price = max(price,0)

    def full_name(self):
```

```

        return f"{self.brand} {self.model_name}"

    def make_a_call(self,number):
        return f"calling {number}...."

class Smartphone(Phone): # derived / child class
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera):
        #two ways
        # Phone.__init__(self,brand,model_name, price) # uncommon way

        super().__init__(brand,model_name, price)
        self.ram = ram
        self.internal_memory = internal_memory
        self.rear_camera = rear_camera

phone = Phone('nokia', '1100', 1000)
smartphone = Smartphone('onePlus', '5', 30000, '6 GB', '64 GB', '20 MP')
print(phone.full_name())
print(smartphone.full_name() + f"and price is {smartphone._price}")

```

Multilevel inheritance , MRO , Method Overriding

```

# can we derive more than one class from base class ?
# multilevel inheritance
# method resolution order (MRO)
# method overriding
# isinstance(), issubclass()

```

```

class Phone: # base class / parent class
    def __init__(self, brand,model_name, price):
        self.brand = brand
        self.model_name = model_name
        self._price = max(price,0)

    def full_name(self):
        return f"{self.brand} {self.model_name}"

    def make_a_call(self,number):
        return f"calling {number}...."

class Smartphone(Phone): # derived / child class
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera):
        #two ways
        # Phone.__init__(self,brand,model_name, price) # uncommon way

        super().__init__(brand,model_name, price)
        self.ram = ram
        self.internal_memory = internal_memory
        self.rear_camera = rear_camera

class Smartphone2(Phone): # derived / child class
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera):
        #two ways
        # Phone.__init__(self,brand,model_name, price) # uncommon way

        super().__init__(brand,model_name, price)
        self.ram = ram
        self.internal_memory = internal_memory
        self.rear_camera = rear_camera

smartphone = Smartphone2('onePlus', '5', 30000, '6 GB', '64 GB', '20 MP')
print(smartphone.full_name())

```

Multilevel inheritance , MRO , Method Overriding

```
# can we derive more than one class from base class ?
# multilevel inheritance
# method resolution order(MRO)
# method overriding
# isinstance(), isinstance()

class Phone: # base class / parent class
    def __init__(self, brand,model_name, price):
        self.brand = brand
        self.model_name = model_name
        self._price = max(price,0)

    def full_name(self):
        return f"{self.brand} {self.model_name}"

    def make_a_call(self,number):
        return f"calling {number}...."

class Smartphone(Phone): # derived / child class
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera):
        #two ways
        # Phone.__init__(self,brand,model_name, price) # uncommon way

        super().__init__(brand,model_name, price)
        self.ram = ram
        self.internal_memory = internal_memory
        self.rear_camera = rear_camera

class Smartphone2(Phone): # derived / child class
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera):
        #two ways
        # Phone.__init__(self,brand,model_name, price) # uncommon way

        super().__init__(brand,model_name, price)
        self.ram = ram
        self.internal_memory = internal_memory
        self.rear_camera = rear_camera

smartphone = Smartphone2('onePlus', '5', 30000, '6 GB', '64 GB', '20 MP')
print(smartphone.full_name())
```

multilevel inheritance

```
class Phone: # base class / parent class
    def __init__(self, brand,model_name, price):
        self.brand = brand
        self.model_name = model_name
        self._price = max(price,0)

    def full_name(self):
        return f"{self.brand} {self.model_name}"

    def make_a_call(self,number):
        return f"calling {number}...."

class Smartphone(Phone): # derived / child class
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera):
        #two ways
```

```

    # Phone.__init__(self,brand,model_name, price) # uncommon way

    super().__init__(brand,model_name, price)
    self.ram = ram
    self.internal_memory = internal_memory
    self.rear_camera = rear_camera

class FlagshipPhone(Smartphone):
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera,
front_camera):
        super().__init__(brand,model_name, price, ram, internal_memory, rear_camera)
        self.front_camera = front_camera

smartphone = Smartphone('onePlus', '5', 30000, '6 GB', '64 GB', '20 MP')
print(smartphone.full_name())

oneplus = FlagshipPhone('onePlus', '5', 30000, '6 GB', '64 GB', '20 MP','18 MP')
print(oneplus.full_name())

```

MRO

```
print(help(FlagshipPhone))
```

Method Overriding

```

class Phone: # base class / parent class
    def __init__(self, brand,model_name, price):
        self.brand = brand
        self.model_name = model_name
        self._price = max(price,0)

    def full_name(self):
        return f"{self.brand} {self.model_name}"

    def make_a_call(self,number):
        return f"calling {number}...."

class Smartphone(Phone): # derived / child class
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera):

        super().__init__(brand,model_name, price)
        self.ram = ram
        self.internal_memory = internal_memory
        self.rear_camera = rear_camera

    def full_name(self):
        return f"{self.brand} {self.model_name} and price is {self._price}"

class FlagshipPhone(Smartphone):
    def __init__(self, brand,model_name, price, ram, internal_memory, rear_camera,
front_camera):
        super().__init__(brand,model_name, price, ram, internal_memory, rear_camera)
        self.front_camera = front_camera

    #def full_name(self):
        #return f"{self.brand} {self.model_name} and price is {self._price} and
front_camera = {self.front_camera}"

```

```

oneplus = FlagshipPhone('onePlus', '5', 30000, '6 GB', '64 GB', '20 MP','18 MP')
print(oneplus.full_name())

```

```

oneplus5 = Smartphone('onePlus', '5', 30000, '6 GB', '64 GB', '20 MP')
oneplus5t = FlagshipPhone('onePlus', '5t', 30000, '6 GB', '64 GB', '20 MP', '16 MP')
# print(isinstance(oneplus5, FlagshipPhone))
# print(isinstance(oneplus5t, Phone))
# print(issubclass(FlagshipPhone, Smartphone))
=====

```

Multiple Inheritance

```

class A:

    def class_a_method(self):
        return 'I\'m just a class A method'

    def hello(self):
        return 'hello from class A'

class B:

    def class_b_method(self):
        return 'I\'m just a class B method'

    def hello(self):
        return 'hello from class B'

class C(A,B):
    pass

instance_c = C()
# print(instance_c.class_a_method())
# print(instance_c.class_b_method())
# print(instance_c.hello())
# print(help(C))
print(C.mro())
print(C.__mro__)
=====
=====
===

```

Intro and built in errors

```

# syntax error

# def func:
#     pass

# indentation errors
# def func():
#     print('hello')
#     print('world')

# name error

# print(func)

```

```

# index error
# l = [1,2,3]
# print(l[4])

#value error
# s = '123'
# print(int(s))

#attribute error

# l = [1,2,3]
# l.push('12')
```

#Key Error

```
d={'name':'Niyam'}
```

```
print(d[age])
```

#Type Error

```
print (5 + 'Nityam')
```

Raise errors

```
def add(a,b):
    if (type(a) is int) and (type(b) is int):
        return a+b
    #return 'OOps you are passing wrong data type to function'
    raise TypeError('OOps you are passing wrong data type to function')

print(add('2','3'))
```

Raise errors another example

```

# raise errors example 1
# NotImplementedError
# abstract method

class Animal:
    def __init__(self,name):
        self.name = name

    def sound(self):
        raise NotImplementedError('you have to define this method in subclasses ')

class Dog(Animal):
    def __init__(self,name,breed):
        super().__init__(name)
        self.breed = breed

    def sound(self):
        return 'bhow bhow'
```

```

class Cat(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

    def sound(self):
        return 'meao meao'

doggy = Dog('boony', 'pug')
print(doggy.sound())

```

Try & Except

```

# Exception handling
# try except else finally

try:
    age = int(input('enter your age: ')) # seven # 7
except ValueError:
    print('Invalid Input')

```

```

if age < 18:
    print('You can\'t play this game.')
else:
    print('you can play this game.')

```

```

# Exception handling
# try except else finally

while True:
    try:
        age = int(input('enter your age: ')) # seven # 7
        break
    except ValueError:
        print('maybe you entered string instead of number , try again')
    except:
        print('unexpected error .. ')

if age < 18:
    print('You can\'t play this game.')
else:
    print('you can play this game.')

```

Else finally with try except

```

while True:
    try:
        number = int(input('enter a number: ')) # seven # 7
        break

    except ValueError:
        print('maybe you entered string instead of number , try again')
    except:
        print('unexpected error .. ')

```

```
else:  
    print(f'input number = {number}')  
    break  
  
finally:  
    print('finally Bocks.....')
```